Example: error in func eval.

sin(1)

sin(x) is implemented in an approximate way (series ...), the eval generates an error.

Condition number: indication of sensitivity of function evaluation with respect to the input.

perturbation

$$\frac{f(x+x\eta)-f(x)}{f(x)} \simeq \frac{xf'(x)}{f(x)}\eta$$

[relative error]

$$\kappa_f(x) = \frac{xf'(x)}{f(x)}$$

Function	Cond. $\left\lvert \frac{x}{x-1} \right\rvert$	Interp
$x-1$	Huge close to 1	Sensitive near 1

Cancellation: $1 + 10^{-16} = 1$

→ $1.00\ldots01 - 1 = 0$

(in double)

→ Loss of information.

→ Difference of two big numbers we risk having large error: catastrophic cancellation

$$p(t) = (t-1)^6$$

$$p(1.001) = (10^{-3})^6 = 10^{-18}$$

using $(t-1)^6$ gives better error results, prevents cancellation errors.

$$x^2 + Bx + 1$$

$$\Delta = b^2 - 4ac = B^2 - 4$$

$$x_1 = \frac{-B - \sqrt{B^2-4}}{2}$$

Ok numerically if B large

$$x_2 = \frac{-B + \sqrt{B^2-4}}{2}$$

Not ok when B is big

$$\sqrt{B^2-4} - B$$

Roughly the same. $\Rightarrow$ lose information.

Fix $\Rightarrow B - \sqrt{B^2-4}$

$$x_2 = \frac{-B + \sqrt{B^2-4}}{2} = \frac{4}{2(-B-\sqrt{B^2-4})}$$

$$= \frac{1}{x_1}$$ $\rightarrow$ Ok numerically

If we want precision guarantees we cannot simply use floating point numbers as they are.

Simply adding more bits to a floating representation does not guarantee the results.

Using only floating point arithmetic cannot control errors.

Using rational numbers can control errors explicitly. But it becomes Expensive for significant applications.

Reason: repeated computations lead to large integers → MEMORY
→ COMPUTATIONS.

Advantages of floating point arithmetic
- use constant memory for large/small numbers
- have controlled cost for math operations

Disadvantages:

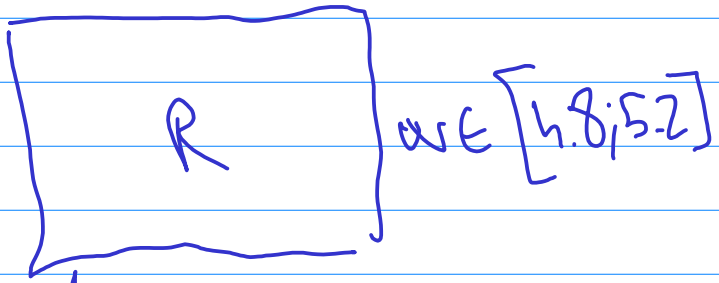
- representation errors (roundoff)
- errors in usual math operations.

Fix: Interval arithmetic.

→ replace a value (fl. point) with an interval (2 fl. pt numbers)

→ learn (code) a way of working with intervals in such a way that the resulting numerical interval is guaranteed to contain the actual result.

Example:



Guaranteed area estimate.

$$\left.\begin{array}{l} 2.9 \le l \le 3.1 \\ 4.8 \le w \le 5.2 \end{array}\right\} \Rightarrow 2.9 \cdot 4.8 \le A \le 3.1 \cdot 5.2$$

$A = l \cdot w$

The resulting interval containing all possible results for variables in the input intervals



→ floating point numbers do not capture all real numbers

→ The set of real numbers can be covered with intervals having FLOATING POINT endpoints (include $\pm\infty$)

[2.9, 3.1]

↓ midpoint

3

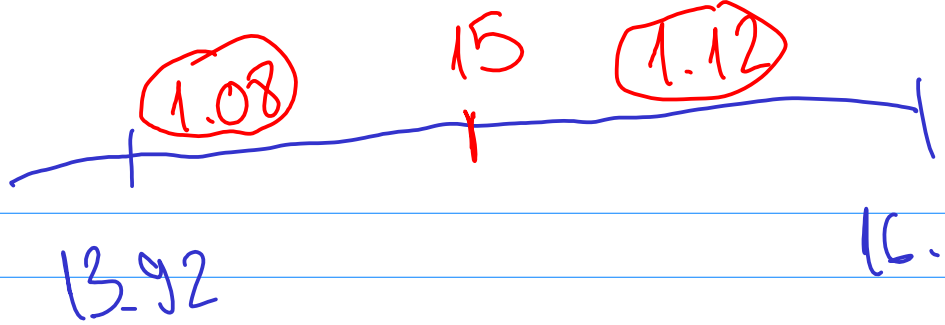
[4.8, 5.2]

↓ mid.

5

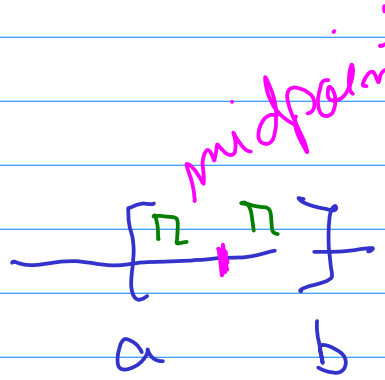
Product of mid points: 15

Real estimate [13.92, 16.12]



Define intervals:

① Endpoints a, b

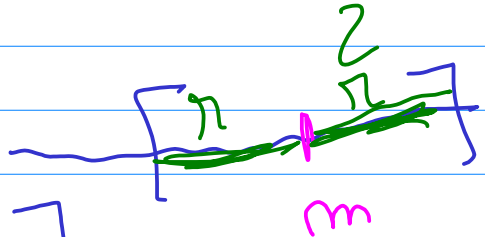


Characteristics: - width: $b-a$

$X=[a,b]$ - midpoint: $\frac{a+b}{2}$

- Radius: $\frac{b-a}{2}$

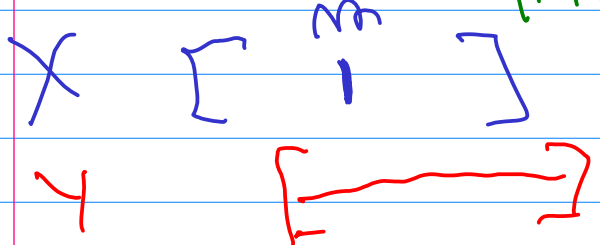
② Midpoint, radius:



$X=[m-r, m+r]$

Membership: $x \in [a,b]$ (number, interval)

Containment $X \subseteq Y$ (interval, interval)



$m \in Y \not\Rightarrow X \subseteq Y$

hull(X,Y): smallest interval containing both X, Y

intersection: usual def

TO DO: look at Notebook 3

→ link on webpage